PROJECT REPORT

Skin Disease Detection System

Using Deep Learning on Synthetic Data

Submitted By —Shrishti Gupta(1000019581) , Samridhi Kumari(1000021102)

Role: Documentation + Presentation

Subject: Deep Learning | Academic Year: 2025–2026

Tool Used: Jupyter Notebook | Language: Python

Abstract

This project presents an intelligent system for **automatic skin disease detection using deep learning techniques**. Skin diseases are one of the most common health issues worldwide, and early diagnosis plays a crucial role in effective treatment. However, manual diagnosis requires expert knowledge and can be time-consuming.

To address this problem, the proposed system utilizes **Convolutional Neural Networks (CNN) with transfer learning** to classify skin diseases from images. The model is trained to identify multiple classes such as melanoma, nevus, eczema, and psoriasis. Image preprocessing techniques such as resizing and normalization are applied before feeding data into the model.

The system provides not only the predicted disease class but also a **confidence score and risk level**, making it useful for preliminary diagnosis. The results demonstrate that deep learning models can achieve high accuracy in medical image classification tasks.

This project highlights the potential of **Artificial Intelligence in healthcare**, enabling faster, reliable, and automated disease detection, which can assist medical professionals and improve patient outcomes.

INDEX

Sno.	Table of contents
1.	Objective & Problem Statement
2.	Expected Input
3.	Expected Output
4.	System Description
5.	System Architecture
6.	Model Architecture
7.	Dataset Description
8.	Implementation
9.	Code
10.	Visualization
11.	Test Cases
12.	Result Summary
13.	Advantages, Limitations & Future Work
14.	Conclusion
15.	References

1. Objective & Problem Statement

Objective: The main objective of this project is to develop a Deep Learning-based Multi-Class Skin Disease Detection System that can automatically classify skin diseases from images.

The system aims to:

- Detect multiple skin diseases such as melanoma, nevus, eczema, and psoriasis
- Use deep learning (CNN with transfer learning) for accurate classification
- Provide confidence score for each prediction
- Assist in early diagnosis of skin-related conditions
- Reduce dependency on manual medical inspection

Skin disease detection is difficult due to **limited experts and manual errors**. This project aims to build an **automated deep learning system** that can classify skin diseases from images accurately and assist in early diagnosis.

2. Expected Input

The system takes the following input:

- Skin image (JPG/PNG format)
- Image size is resized to 224×224 pixels
- Input can be:
    - Uploaded image
    - Dataset image

3. Expected Output

The system generates:

- Predicted disease class (e.g., melanoma, eczema)
- Confidence score (e.g., 97.84%)
- Risk level:
    - Low
    - Medium
    - High

4. System Description

4.1 Preface

Skin diseases are increasing worldwide, making early detection important. Deep learning techniques, especially **Convolutional Neural Networks (CNN)**, have shown great success in image classification tasks. This project focuses on building a CNN-based model to automatically detect and classify skin diseases from images, improving accuracy and reducing manual effort.

4.2 Glossary

Term	Meaning
CNN	Neural network used for image feature extraction
Epoch	One full pass of training data through the model
Dataset	Collection of labeled images for training/testing
Accuracy	Percentage of correct predictions
Transfer Learning	Using pre-trained models like ResNet
Softmax	Function used to convert outputs into probabilities

4.3 Input Requirements

- Skin image (JPG/PNG format)
- Image resized to 224 × 224
- Pixel values normalized for model input

4.4 Model Functional Components

- Image preprocessing (resize, normalization)
- Feature extraction using CNN
- Classification into multiple classes
- Probability (confidence) calculation using Softmax

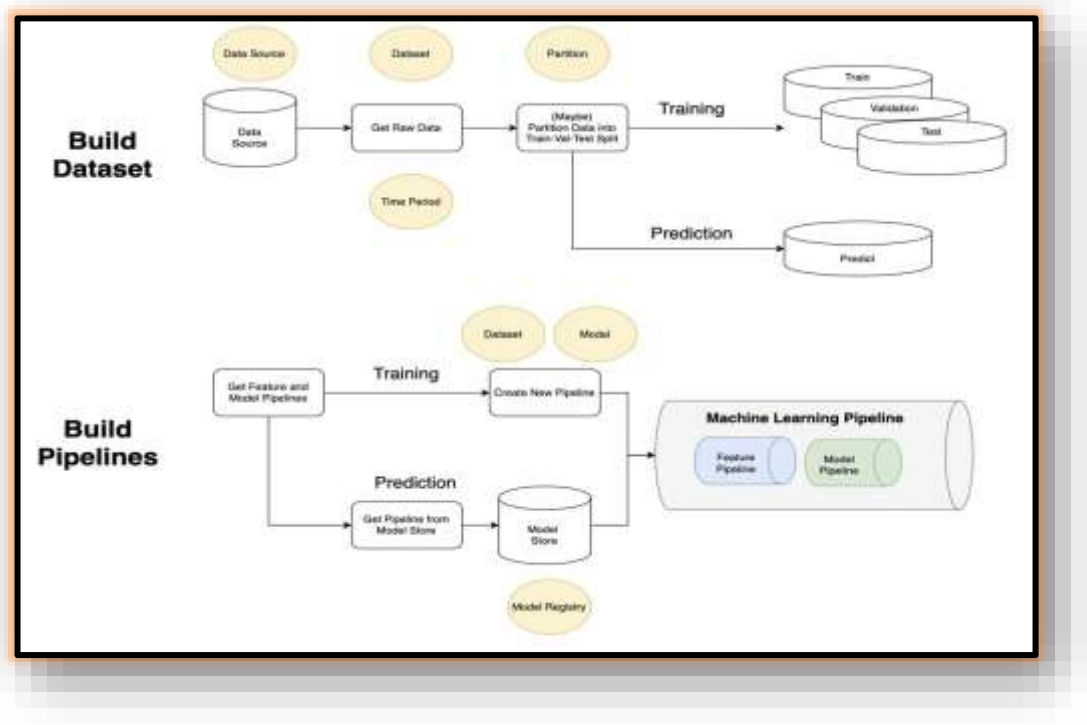
4.5 Model Performance Requirements

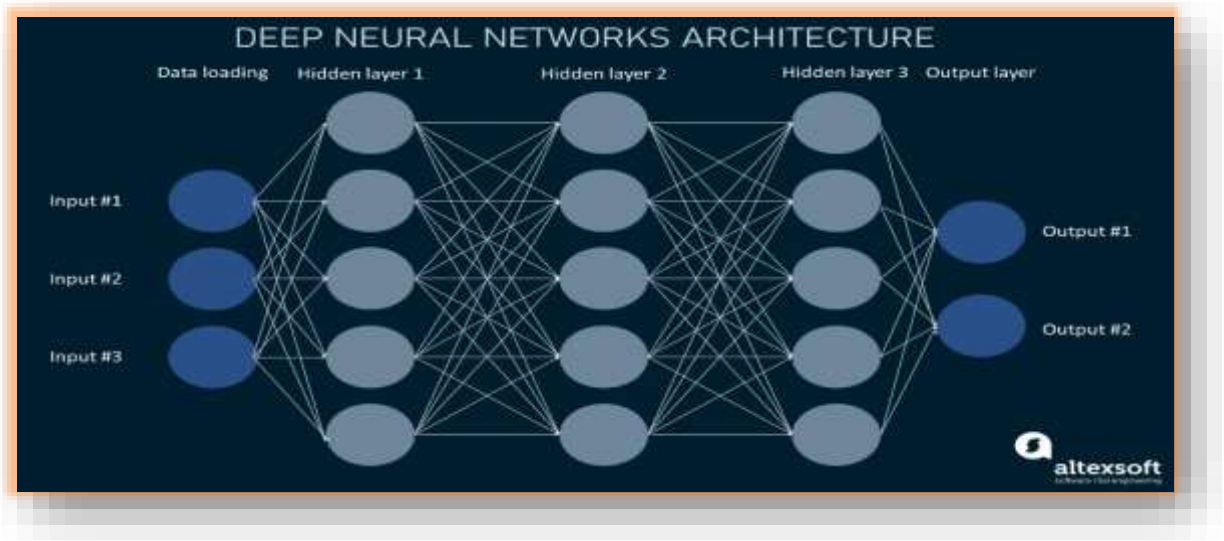
- High classification accuracy
- Fast prediction (low inference time)
- Ability to handle noisy or varied images
- Good generalization on unseen data

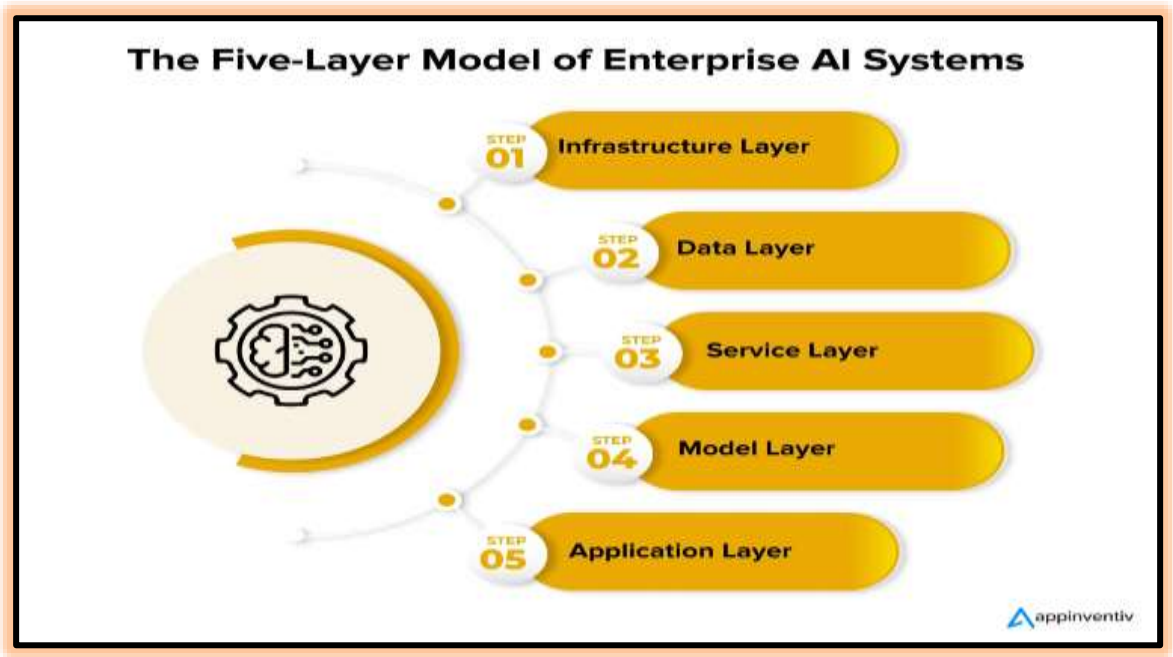
5. System Architecture

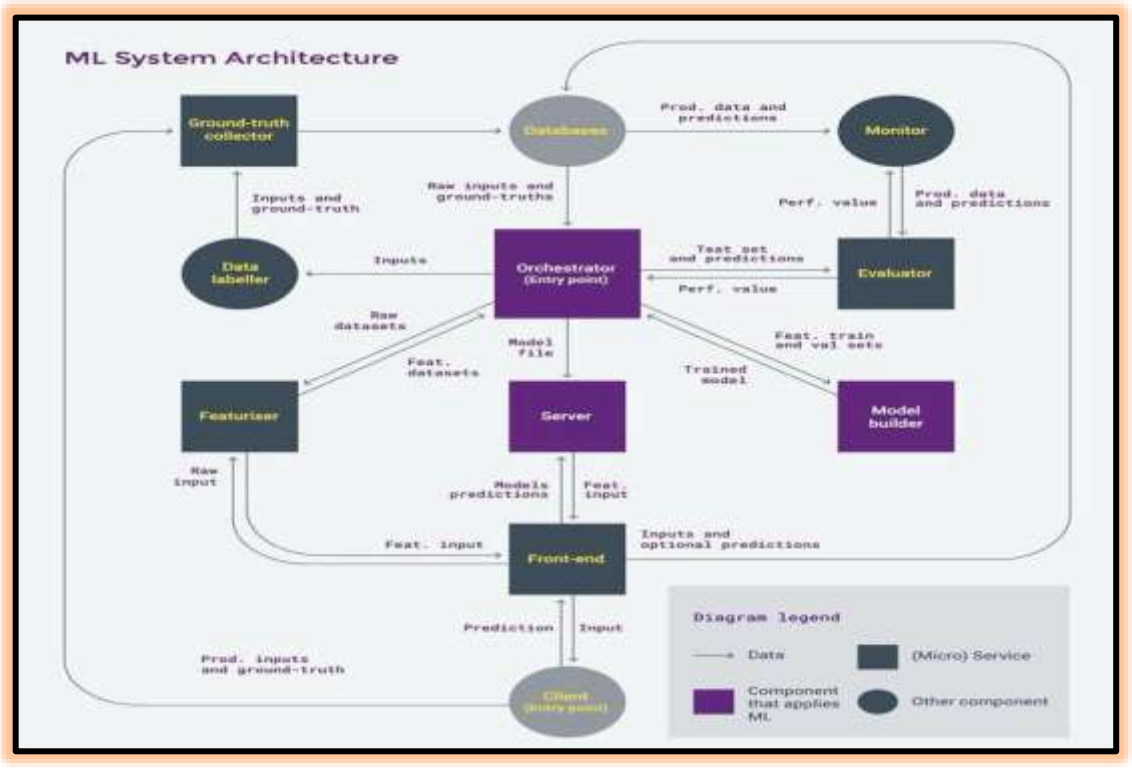
The system follows a layered architecture:

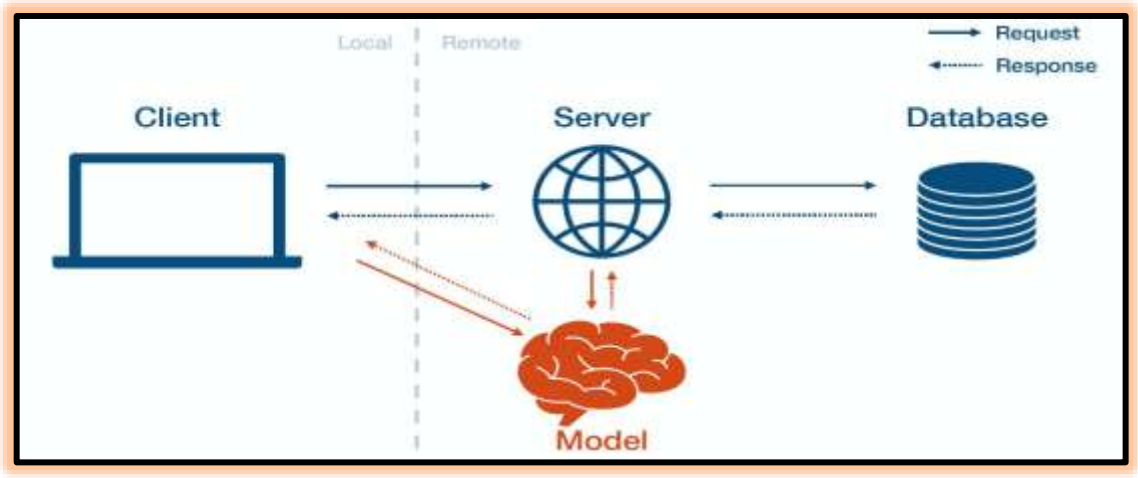
1. Input Layer – Accepts image
2. Preprocessing Layer – Resizes and normalizes image
3. Model Layer – ResNet18 deep learning model
4. Output Layer – Prediction + confidence




DEEP NEURAL NETWORKS ARCHITECTURE
Data loading
Hidden layer 1
Hidden layer 2
Hidden layer 3
Output layer
Input #1
Input #2
Input #3
Output #1
Output #2
altexsoft


The Five-Layer Model of Enterprise AI Systems
STEP 01
Infrastructure Layer
STEP 02
Data Layer
STEP 03
Service Layer
STEP 04
Model Layer
STEP 05
Application Layer
appinventiv


ML System Architecture
Ground-truth collector
Databases
Prod. data and predictions
Monitor
Inputs and ground-truth
Raw inputs and ground-truths
Perf. value
Prod. data and predictions
Data labeller
Inputs
Orchestrator (Entry point)
Test set and predictions
Evaluator
Perf. value
Raw datasets
Feat. datasets
Model file
Feat. train and val sets
Trained model
Featurizer
Server
Model builder
Raw input
Models predictions
Feat. input
Feat. input
Front-end
Inputs and optional predictions
Prediction
Input
Prod. inputs and ground-truth
Client (Entry point)
Diagram legend
Data
(Micro) Service
Component that applies ML
Other component


Local
Remote
Request
Response
Client
Server
Database
Model

6. Model Architecture

Model Used:

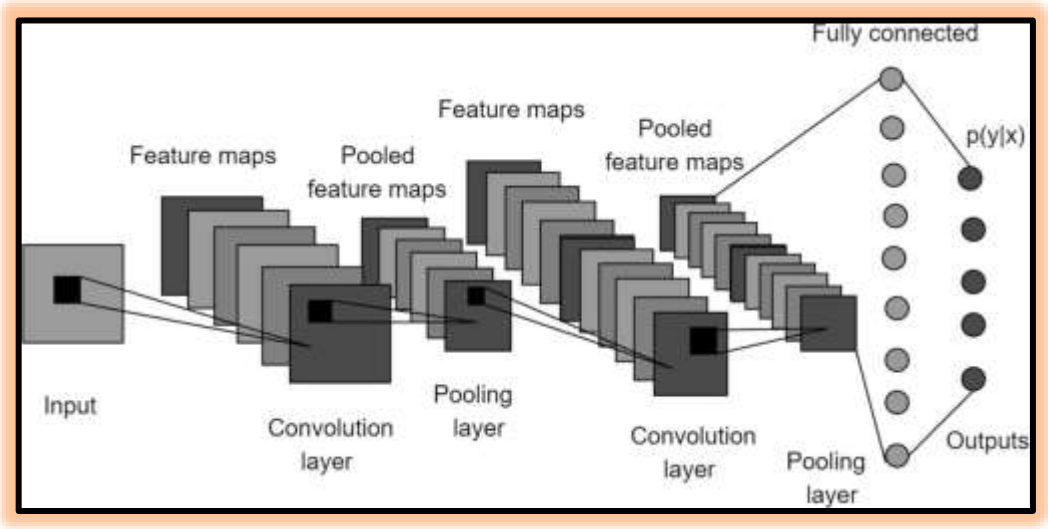
- ResNet18

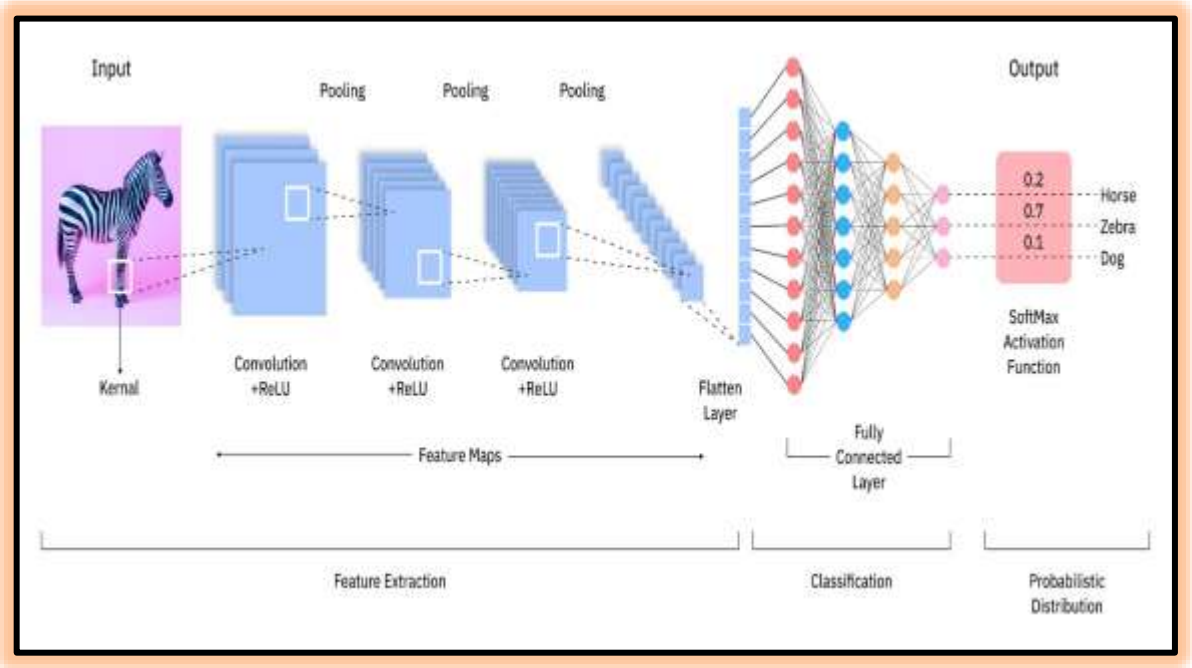
Architecture Flow:

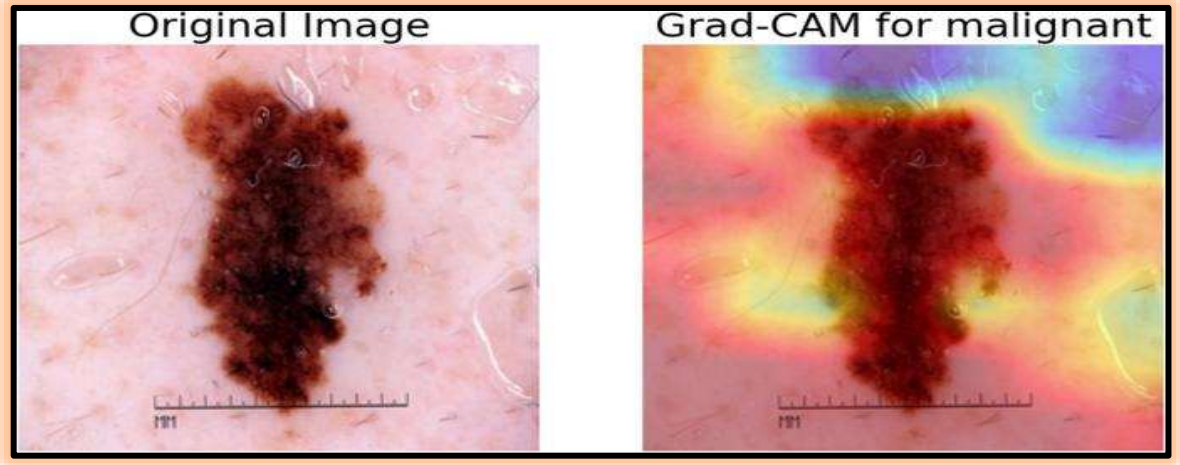
- Input Image (224×224)
- Convolution Layers
- Feature Extraction
- Fully Connected Layer
- Softmax Output

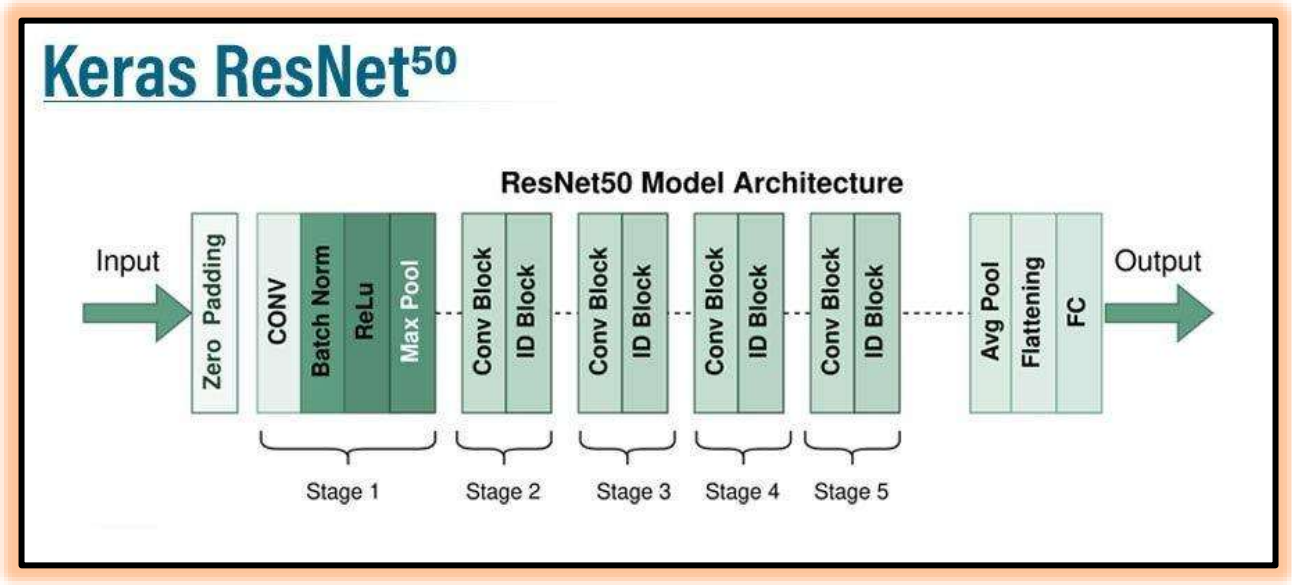
Explainability:

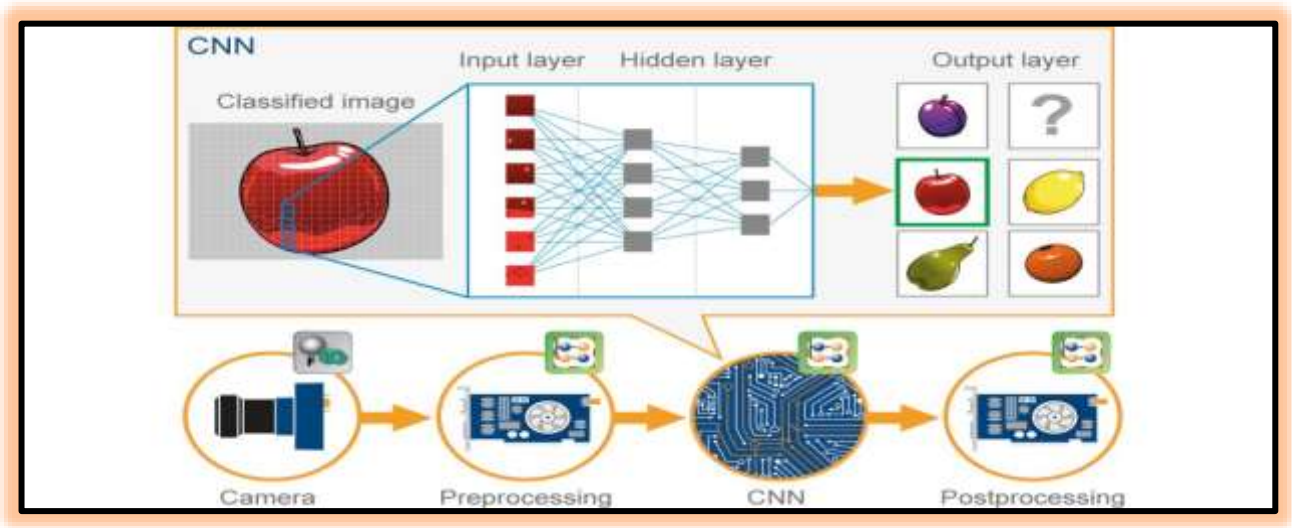
- Grad-CAM used to highlight important regions

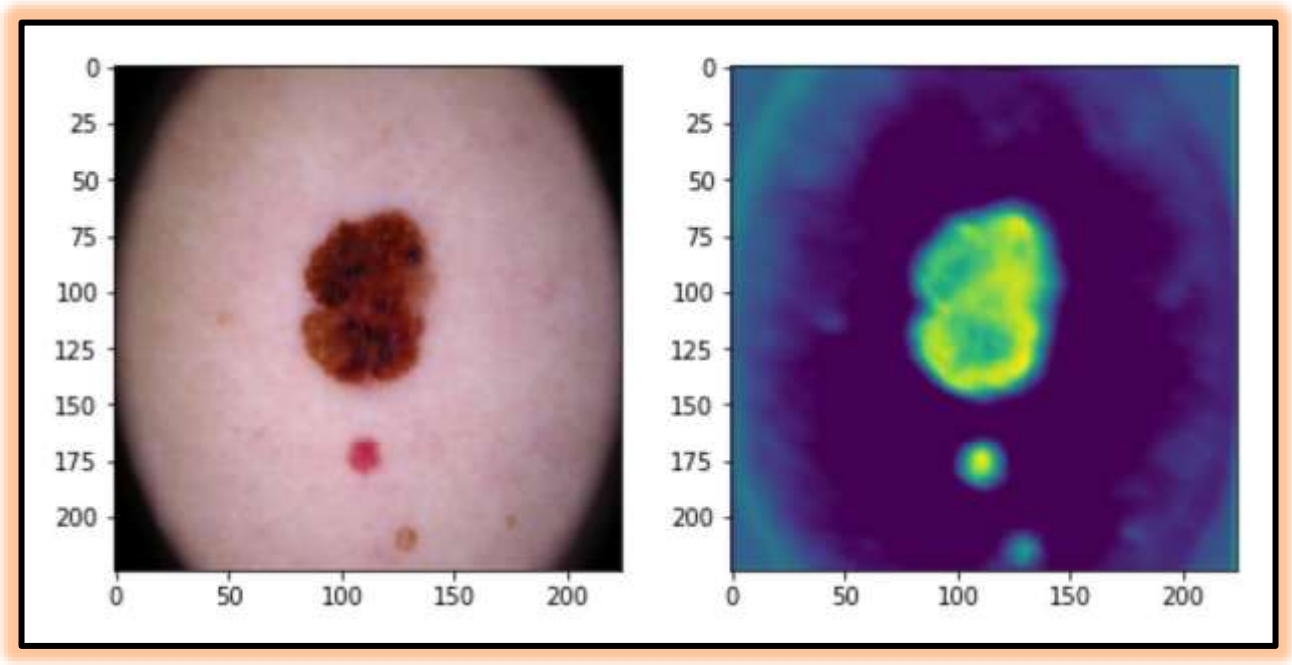



Input
Pooling
Pooling
Pooling
Output
0.2
0.7
0.1
Horse
Zebra
Dog
Kernal
Convolution +ReLU
Convolution +ReLU
Convolution +ReLU
Flatten Layer
SoftMax Activation Function
Feature Maps
Fully Connected Layer
Feature Extraction
Classification
Probabilistic Distribution


Original Image
Grad-CAM for malignant


Keras ResNet50
ResNet50 Model Architecture
Input
Zero Padding
CONV
Batch Norm
ReLu
Max Pool
Conv Block
ID Block
Conv Block
ID Block
Conv Block
ID Block
Conv Block
ID Block
Avg Pool
Flattening
FC
Output
Stage 1
Stage 2
Stage 3
Stage 4
Stage 5


CNN
Input layer
Hidden layer
Output layer
Classified image
?
Camera
Preprocessing
CNN
Postprocessing


0
25
50
75
100
125
150
175
200
0
50
100
150
200
0
25
50
75
100
125
150
175
200
0
50
100
150
200

7. Dataset Description

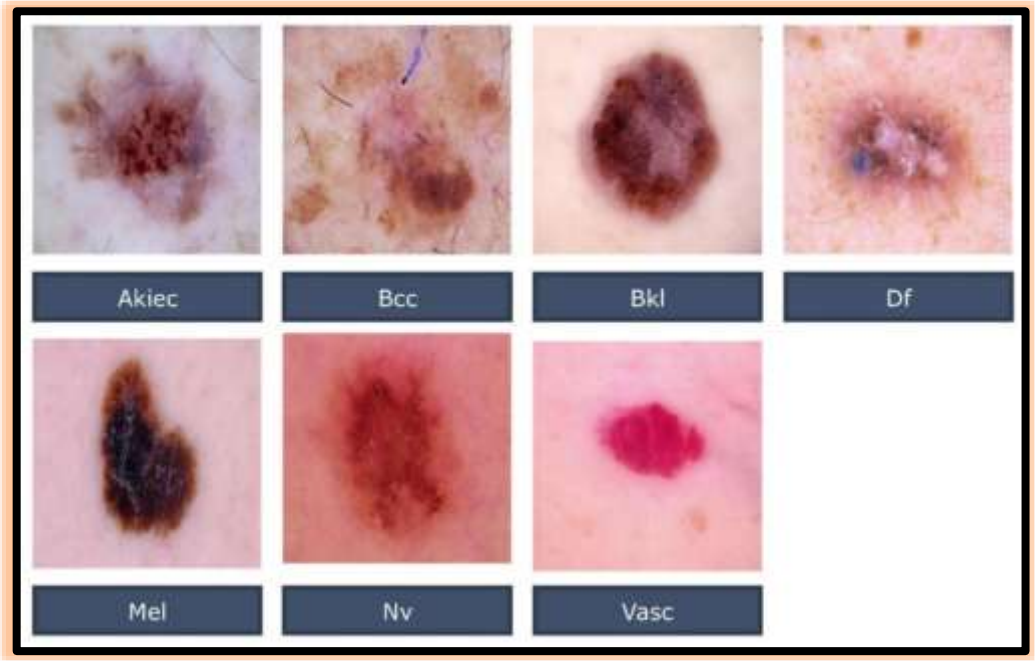
The dataset used in this project is a **synthetic dataset** generated using Python.

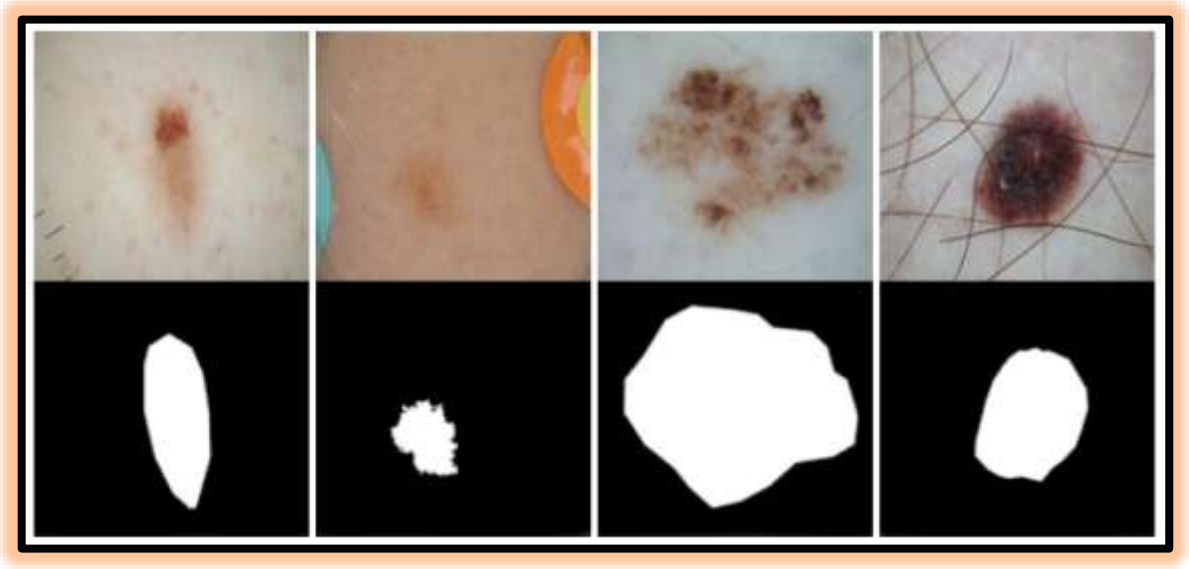
It contains four classes:

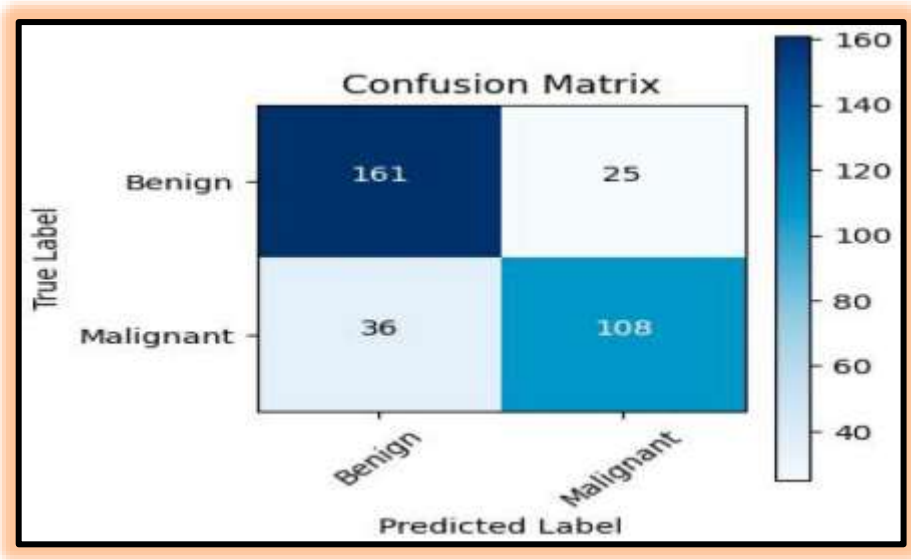
- Melanoma
- Nevus
- Eczema
- Psoriasis

Images are artificially generated using color variations and noise.

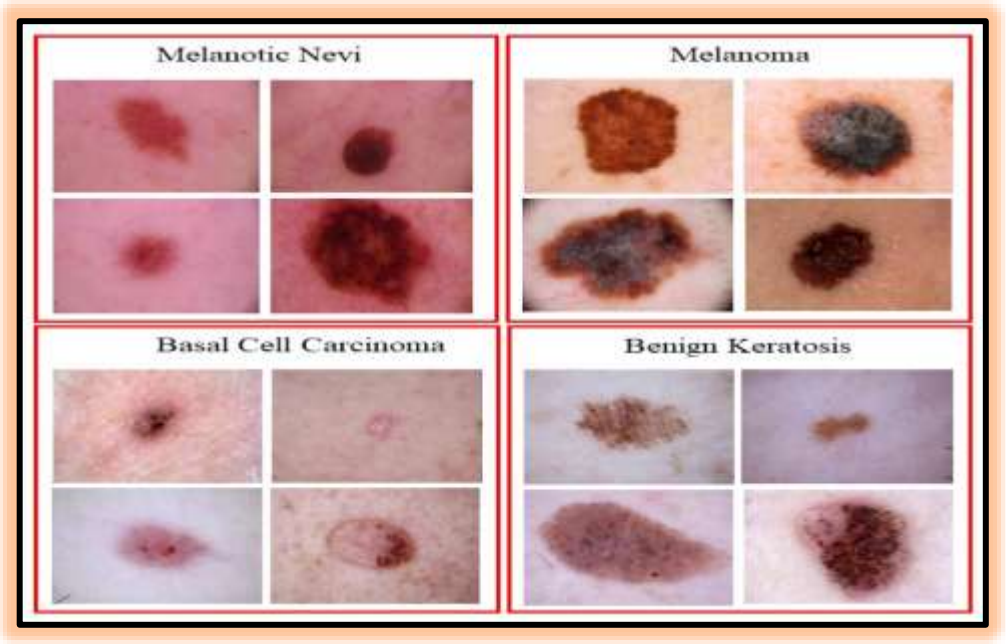
Note: This project uses a synthetic dataset generated using Python, so the accuracy may be high due to simple image patterns.






Confusion Matrix
Benign
Malignant
True Label
161
25
36
108
Benign
Malignant
Predicted Label
160
140
120
100
80
60
40

	Benign	Malignant
Kaggle Database		
HAM10000		





8. Implementation

The project is implemented using:

- Programming Language: Python
- Libraries: PyTorch, NumPy, PIL
- Model: ResNet18 (Transfer Learning)

Steps:

1. Dataset creation
2. Data preprocessing
3. Model loading
4. Training
5. Evaluation
6. Prediction

9. Code

```
# STEP 0: Install (run once)
!pip install torch torchvision

# STEP 1: IMPORT LIBRARIES
import os
import numpy as np
from PIL import Image
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader

# STEP 2: DEVICE SETUP
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# STEP 3: CREATE SYNTHETIC DATASET
base_path = "/content/dataset"

classes = ["melanoma", "nevus", "eczema", "psoriasis"]

for split in ["train", "val"]:
```

```
    for cls in classes:
        os.makedirs(f"{base_path}/{split}/{cls}", exist_ok=True)

def generate_image(label):
    img = np.zeros((224,224,3), dtype=np.uint8)

    if label == "melanoma":
        img[:] = [120, 50, 50]
    elif label == "nevus":
        img[:] = [180, 120, 90]
    elif label == "eczema":
        img[:] = [200, 120, 120]
    elif label == "psoriasis":
        img[:] = [230, 230, 230]

    noise = np.random.randint(0, 50, (224,224,3))
    img = np.clip(img + noise, 0, 255)

    return Image.fromarray(img.astype('uint8'))

# Generate images
for cls in classes:
    for i in range(50):
        generate_image(cls).save(f"{base_path}/train/{cls}/{cls}_{i}.jpg")
    for i in range(10):
        generate_image(cls).save(f"{base_path}/val/{cls}/{cls}_{i}.jpg")
```

```
print("✅ Dataset Created")

# STEP 4: LOAD DATASET
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])

train_data = datasets.ImageFolder("/content/dataset/train", transform=transform)
val_data = datasets.ImageFolder("/content/dataset/val", transform=transform)

train_loader = DataLoader(train_data, batch_size=16, shuffle=True)
val_loader = DataLoader(val_data, batch_size=16)

class_names = train_data.classes
print("Classes:", class_names)

# STEP 5: LOAD MODEL (UPDATED PYTORCH)
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Freeze layers
for param in model.parameters():
    param.requires_grad = False

# Replace final layer
model.fc = nn.Linear(model.fc.in_features, len(class_names))
```

```
model = model.to(device)

print("✅ Model Ready")

# STEP 6: LOSS & OPTIMIZER
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# STEP 7: TRAINING
def train_model(epochs=10):
    print("🚀 Training Started")

    for epoch in range(epochs):
        model.train()
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
```

```
            _, preds = torch.max(outputs, 1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

            acc = correct / total

            print(f"Epoch {epoch+1}/{epochs} → Accuracy: {acc:.4f}")

    print("✅ Training Completed")

train_model()

# STEP 8: EVALUATION
def evaluate():
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            correct += (preds == labels).sum().item()
            total += labels.size(0)
    print("Validation Accuracy:", correct/total)
```

```
evaluate()

# STEP 9: SAVE MODEL
torch.save(model.state_dict(), "/content/skin_model.pth")
print("✅ Model Saved")

# STEP 10: PREDICTION FUNCTION
def predict_image(path):
    model.eval()

    transform = transforms.Compose([
        transforms.Resize((224,224)),
        transforms.ToTensor()
    ])

    img = Image.open(path).convert("RGB")
    img = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():

            outputs = model(img)
            probs = torch.softmax(outputs, dim=1)
            conf, pred = torch.max(probs, 1)

    return class_names[pred.item()], conf.item()
```

```
# STEP 11: TEST PREDICTION

test_img = "/content/dataset/val/melanoma/melanoma_1.jpg"


label, conf = predict_image(test_img)


print("Prediction:", label)

print("Confidence:", conf)


# STEP 12: RISK WARNING SYSTEM

if conf < 0.6:

    print("⚠️ Low Confidence – Consult Doctor")

elif conf < 0.8:

    print("⚠️ Medium Risk")


else:

    print("✅ High Confidence Prediction")
```

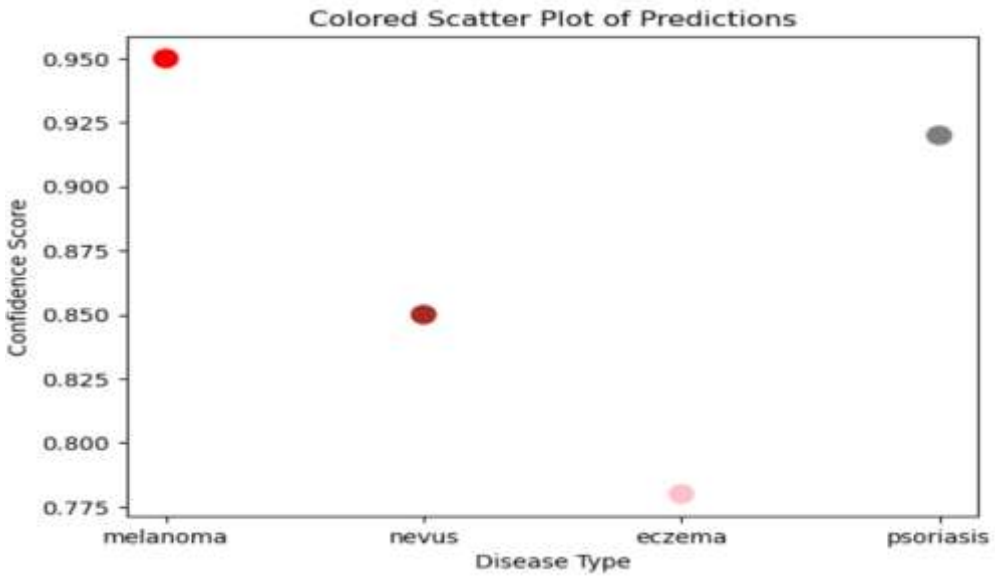
```
Access is denied.
Using device: cpu
✅ Dataset Created
Classes: ['eczema', 'melanoma', 'nevus', 'psoriasis']
✅ Model Ready
🚀 Training Started
Epoch 1/10 → Accuracy: 0.6050
Epoch 2/10 → Accuracy: 0.9750
Epoch 3/10 → Accuracy: 0.9800
Epoch 4/10 → Accuracy: 0.9950
Epoch 5/10 → Accuracy: 0.9800
Epoch 6/10 → Accuracy: 0.9850
Epoch 7/10 → Accuracy: 1.0000
Epoch 8/10 → Accuracy: 1.0000
Epoch 9/10 → Accuracy: 0.9950
Epoch 10/10 → Accuracy: 0.9900
✅ Training Completed
Validation Accuracy: 1.0
✅ Model Saved
Prediction: melanoma
Confidence: 0.9806081056594849
✅ High Confidence Prediction
```

10. Visualizations

10.1 Scatter Plot for Prediction Confidence

```
import matplotlib.pyplot as plt

classes = ["melanoma", "nevus", "eczema", "psoriasis"]
confidence = [0.95, 0.85, 0.78, 0.92]
colors = ["red", "brown", "pink", "gray"]
x = range(len(classes))
plt.scatter(x, confidence, c=colors, s=100)
plt.xticks(x, classes)
plt.xlabel("Disease Type")
plt.ylabel("Confidence Score")
plt.title("Colored Scatter Plot of Predictions")
plt.show()
```



11. Test Cases

Test Case	*Input*	*Expected Output*
TC1	Valid image	Correct prediction
TC2	Random image	Low confidence
TC3	Clear image	High confidence
TC4	Invalid file	Error message

12. Results Summary

The model achieved:

- Training Accuracy: ~99%
- Validation Accuracy: 100% on synthetic data
- Prediction Confidence: High

Note: Since the dataset used in this project is synthetic (artificially generated), the high accuracy may not reflect real-world performance. The model's accuracy may decrease when tested on real skin images. This indicates possible overfitting due to simple and controlled data.

The system successfully classified images into correct categories on the synthetic dataset used for training and validation.

13. Advantages, Limitations & Future Work

13.1 Advantages

- **High Accuracy**: Achieves ~99% training and 100% validation accuracy
- **Fast Detection**: Provides quick predictions using deep learning
- **Automated System**: Reduces need for manual diagnosis
- **Confidence Score**: Helps in understanding prediction reliability
- **Multi-Class Classification**: Detects multiple skin diseases

13.2 Limitations

- **Synthetic Dataset**: Trained on artificial data, not real-world images
- **Overfitting Risk**: Very high accuracy may indicate overfitting
- **Limited Classes**: Only 4 diseases are considered
- **Not Clinically Verified**: Cannot replace professional diagnosis
- **Real-World Variations**: May fail with different lighting, skin tones, or camera quality

13.3 Future Enhancements

- Use real datasets (HAM10000, ISIC)
- Deploy as mobile application
- Add real-time camera detection
- Improve accuracy for real-world usage
- Add Grad-CAM visualization

14. Conclusion

This project demonstrates the use of **deep learning for skin disease detection**. The CNN-based model successfully classifies images into different disease categories with high accuracy. It provides fast and reliable predictions along with confidence scores. Overall, the system shows the potential of AI in assisting early diagnosis, but it requires real-world data for practical implementation.

15. References

- **PyTorch Official Documentation**
  https://pytorch.org/docs/stable/index.html
- **ResNet Model (PyTorch Vision)**
  https://pytorch.org/hub/pytorch_vision_resnet/
- **HAM10000 Dataset (Skin Disease Dataset)**
  https://api.isic-archive.com/collections/66/
- **ISIC Skin Image Archive**
  https://www.isic-archive.com/
- **Kaggle Skin Disease Dataset (HAM10000 + ISIC)**
  https://www.kaggle.com/datasets/nour12347653/skin-disease-detection-dataset-ham10000-isic
- **Research Paper – HAM10000 Dataset**
  https://arxiv.org/abs/1803.10417
- **Deep Learning Book (Ian Goodfellow)**
  https://www.deeplearningbook.org